

PROJECT DOCUMENTATION

EduGenie: Google Gemini Powered Learning Assistant

AI-powered learning assistant for questions, explanations, summaries, quizzes and learning paths

Submitted By	Anadhavalli A
College	Sri Ram Nallamani Yadava College of Arts and Science
Department	B.Sc. AI & DS
Date	25/09/2026

Mentor: _____

1. Project Description

EduGenie is a lightweight AI-powered educational assistant that simplifies learning through generative AI. It enables learners to ask questions, understand complex concepts, generate quizzes, receive personalized learning recommendations and summarize educational passages.

- Ask questions and receive smart, concise answers.
- Understand complex concepts through simplified explanations.
- Generate quizzes from topics or text.
- Receive personalized learning recommendations.
- Summarize large educational passages.

2. Project Scenarios

1. A student asks EduGenie a general academic question and receives an answer.
2. A student uses the Explain feature to understand a concept.
3. A learner provides a long paragraph and receives a summary.
4. A learner generates a multiple-choice quiz from a topic or passage.
5. A learner requests a structured path from beginner to advanced level.

3. Objectives

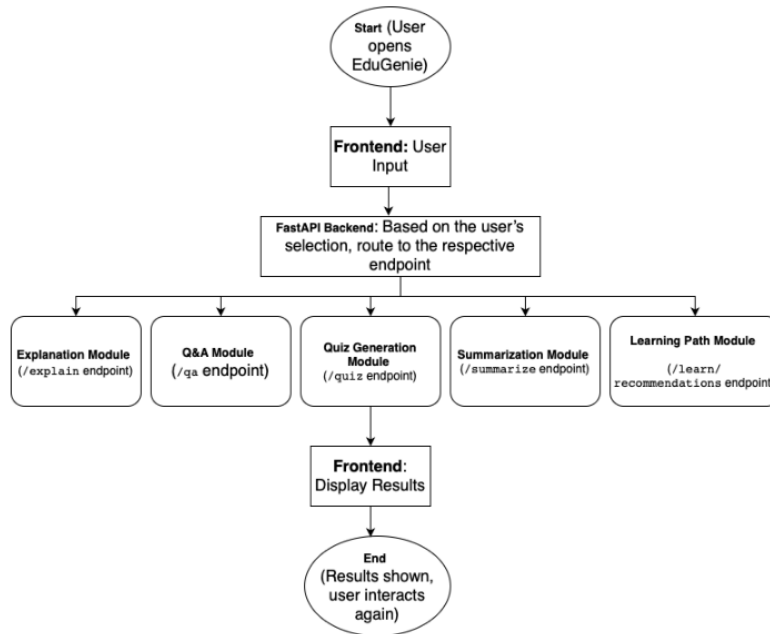
- Provide a simple AI-based educational interface.
- Integrate Google Gemini into practical learning workflows.
- Separate learning functions into modular Python components.
- Support question answering, explanations, summaries and quizzes.
- Generate structured learning recommendations.

4. System Architecture

The application follows a modular architecture in which the frontend accepts user input, the FastAPI backend routes the request to the appropriate module, and the AI client generates the response.



ARCHITECTURAL DIAGRAM



Pre-requisites

1. Python 3.10+
 - Official Documentation: [Google AI for Developers](#)
 - Installation Guide: [Tom's Hardware](#)
 - Popular Tutorial: [Python Programming Tutorial - Full Course for Beginners](#)
 - Installation Steps:
 1. Download the latest Python 3.10+ installer for your operating system from the official website.
 2. Run the installer and ensure you check the box that says "Add Python to PATH".
 3. Follow the installation prompts to complete the setup.
 4. Verify the installation by opening a terminal or command prompt and typing `python --version`.
2. FastAPI Framework –
 - Official Documentation: [FastAPI](#)
 - User Guide: [FastAPI](#)
 - Popular Tutorial: [FastAPI Crash Course](#)

Fig. Architecture diagram from the supplied EduGenie reference document

5. Prerequisites and Technologies

Technology	Purpose
Python 3.10+	Backend and AI module development.
FastAPI	REST-style backend API.
Uvicorn	ASGI server for local execution.
Jinja2	HTML templating.
HTML & CSS	Frontend structure and styling.
Google Gemini API	Generative AI service used by the AI client.

6. Project Workflow

6. Model selection and architecture definition.
7. Module implementation for explanation, Q&A, quiz, summary and learning path.
8. FastAPI backend endpoint development.
9. Frontend integration with the backend.
10. Local deployment and functional testing.

7. Project Folder Architecture

```
EduGenie/  
├── api/  
├── static/  
├── templates/  
├── tests/  
├── ai_client.py  
├── explanation_module.py  
├── learning_path.py  
├── main.py  
├── qna.py  
├── quiz_module.py  
├── summary_module.py  
├── requirements.txt  
└── run_windows.bat
```

Fig. Current project structure

8. Module Implementation

8.1 Explanation Module

Provides simplified explanations for learner-selected topics.

8.2 Q&A Module

Processes user questions and returns AI-generated answers.

8.3 Quiz Module

Generates multiple-choice questions from a topic or passage.

8.4 Summary Module

Produces concise versions of longer text.

8.5 Learning Path Module

Generates structured learning recommendations from beginner to advanced.

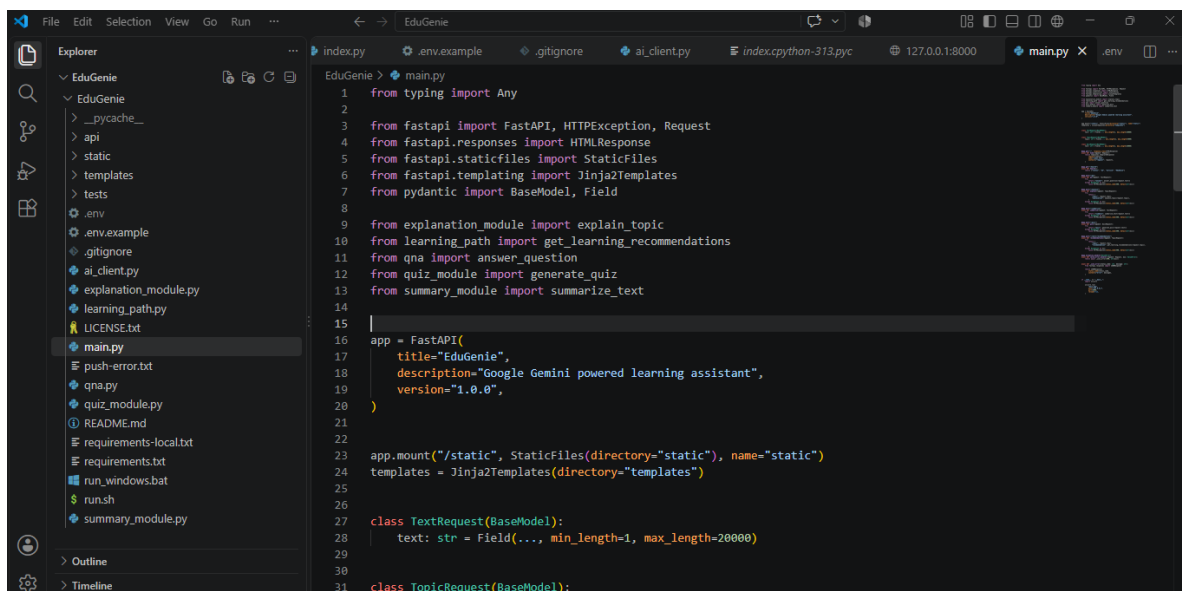
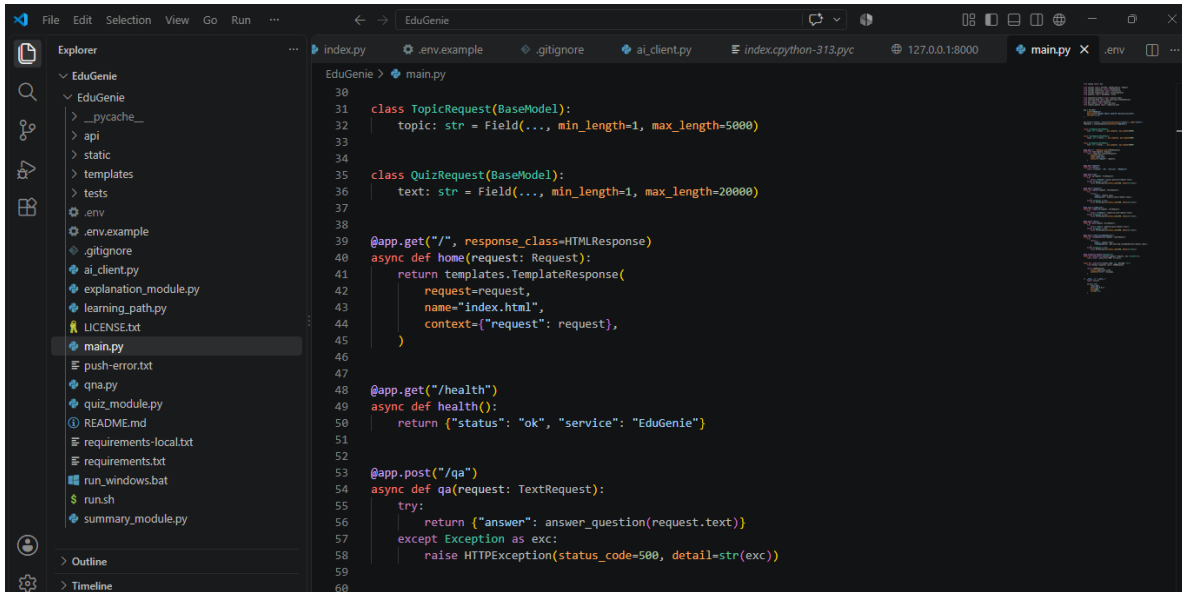
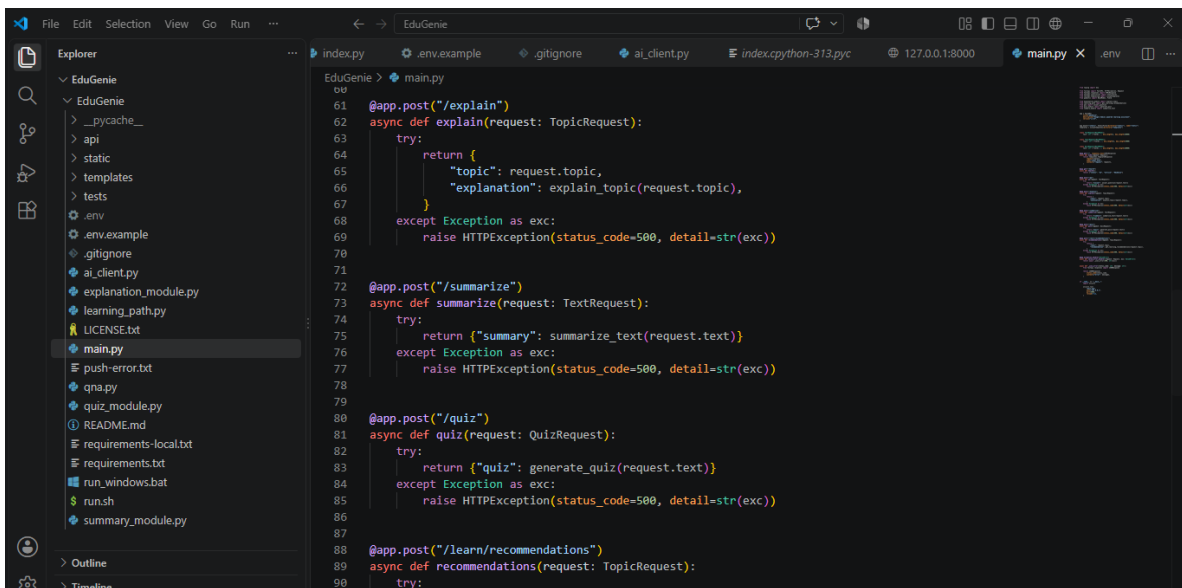


Fig. main.py – imports, FastAPI application and module connections



```
30
31 class TopicRequest(BaseModel):
32     topic: str = Field(..., min_length=1, max_length=5000)
33
34
35 class QuizRequest(BaseModel):
36     text: str = Field(..., min_length=1, max_length=20000)
37
38
39 @app.get("/", response_class=HTMLResponse)
40 async def home(request: Request):
41     return templates.TemplateResponse(
42         request=request,
43         name="index.html",
44         context={"request": request},
45     )
46
47
48 @app.get("/health")
49 async def health():
50     return {"status": "ok", "service": "EduGenie"}
51
52
53 @app.post("/qa")
54 async def qa(request: TextRequest):
55     try:
56         return {"answer": answer_question(request.text)}
57     except Exception as exc:
58         raise HTTPException(status_code=500, detail=str(exc))
59
60
```

Fig. main.py – request models and Q&A endpoint



```
61 @app.post("/explain")
62 async def explain(request: TopicRequest):
63     try:
64         return {
65             "topic": request.topic,
66             "explanation": explain_topic(request.topic),
67         }
68     except Exception as exc:
69         raise HTTPException(status_code=500, detail=str(exc))
70
71
72 @app.post("/summarize")
73 async def summarize(request: TextRequest):
74     try:
75         return {"summary": summarize_text(request.text)}
76     except Exception as exc:
77         raise HTTPException(status_code=500, detail=str(exc))
78
79
80 @app.post("/quiz")
81 async def quiz(request: QuizRequest):
82     try:
83         return {"quiz": generate_quiz(request.text)}
84     except Exception as exc:
85         raise HTTPException(status_code=500, detail=str(exc))
86
87
88 @app.post("/learn/recommendations")
89 async def recommendations(request: TopicRequest):
90     try:
91
```

Fig. main.py – Explain, Summarize and Quiz endpoints

The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the main.py file open in the center. The file explorer shows a project named 'EduGenie' with various files and folders. The main.py file contains the following code:

```
88 @app.post("/learn/recommendations")
89 async def recommendations(request: TopicRequest):
90     try:
91         return {
92             "topic": request.topic,
93             "recommendation": get_learning_recommendations(request.topic),
94         }
95     except Exception as exc:
96         raise HTTPException(status_code=500, detail=str(exc))
97
98
99 @app.exception_handler(ValueError)
100 async def value_error_handler(request: Request, exc: ValueError):
101     return await _json_error(400, str(exc))
102
103
104 async def _json_error(status_code: int, message: str):
105     from fastapi.responses import JSONResponse
106
107     return JSONResponse(
108         status_code=status_code,
109         content={"error": message},
110     )
111
112
113 if __name__ == "__main__":
114     import uvicorn
115
116     uvicorn.run(
117         "main:app",
118         host="127.0.0.1",
```

Fig. main.py – Learning recommendations and error handling

The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the main.py file open in the center. The file explorer shows a project named 'EduGenie' with various files and folders. The main.py file contains the following code:

```
104 async def _json_error(status_code: int, message: str):
105     from fastapi.responses import JSONResponse
106
107     return JSONResponse(
108         status_code=status_code,
109         content={"error": message},
110     )
111
112
113 if __name__ == "__main__":
114     import uvicorn
115
116     uvicorn.run(
117         "main:app",
118         host="127.0.0.1",
119         port=8000,
120         reload=True,
121     )
```

Fig. main.py – Uvicorn server configuration

```
1 """Shared Gemini client and helpers for EduGenie."""
2
3 import json
4 import os
5 import re
6 import time
7 from typing import Any
8
9 from dotenv import load_dotenv
10 from google import genai
11 from google.genai import types
12
13
14 # Load variables from .env
15 load_dotenv()
16
17 # Use the model from .env.
18 # If GEMINI_MODEL is missing, use the lightweight model.
19 MODEL_NAME = os.getenv("GEMINI_MODEL", "gemini-3.5-flash-lite")
20
21
22 def _get_api_key() -> str:
23     """Get the Gemini API key from environment variables."""
24
25     key = os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY")
26
27     if not key:
28         raise RuntimeError(
29             "Gemini API key is not configured. "
30             "Set GEMINI_API_KEY in your environment or .env file."
31         )
```

Fig. ai_client.py – Gemini client setup and model configuration

```
23 def _get_api_key() -> str:
24     if not key:
25         raise RuntimeError(
26             "Gemini API key is not configured. "
27             "Set GEMINI_API_KEY in your environment or .env file."
28         )
29     return key
30
31
32 def get_client() -> genai.Client:
33     """Create and return a Gemini client."""
34     return genai.Client(api_key=_get_api_key())
35
36
37 def _generate_with_retry(
38     prompt: str,
39     *,
40     system_instruction: str | None = None,
41     temperature: float = 0.4,
42     max_output_tokens: int = 2048,
43     response_mime_type: str | None = None,
44 ):
45     """Generate Gemini content with retries for temporary errors."""
46
47     client = get_client()
48
49     config = types.GenerateContentConfig(
50         temperature=temperature,
51         max_output_tokens=max_output_tokens,
```

Fig. ai_client.py – Gemini generation and retry configuration

```

50 ):
51     config = types.GenerateContentConfig(
52         system_instruction=system_instruction,
53         response_mime_type=response_mime_type,
54     )
55
56     # Try the configured model first, then a lightweight fallback.
57     models_to_try = [
58         MODEL_NAME,
59         "gemini-3.5-flash-lite",
60     ]
61
62     # Remove duplicates while keeping order.
63     models_to_try = list(dict.fromkeys(models_to_try))
64
65     last_error = None
66
67     for model in models_to_try:
68         for attempt in range(3):
69             try:
70                 response = client.models.generate_content(
71                     model=model,
72                     contents=prompt,
73                     config=config,
74                 )
75
76                 text = getattr(response, "text", None)
77
78                 if not text:
79                     raise RuntimeError(
80                         "Gemini returned an empty response."
81                     )
82
83             except Exception as exc:
84                 last_error = exc
85
86                 error_text = str(exc).lower()
87
88                 temporary_error = any(
89                     code in error_text
90                     for code in [
91                         "429",
92                         "500",
93                         "503",
94                         "504",
95                         "unavailable",
96                         "overloaded",
97                         "high demand",
98                         "temporarily",
99                     ]
100                 )
101
102                 if not temporary_error:
103                     raise
104
105                 # Exponential backoff: 2s, 4s, 8s
106                 if attempt < 2:
107                     time.sleep(2 ** (attempt + 1))
108
109             return response
110
111     raise last_error
112
113
114
115

```

Fig. ai_client.py – retry handling for temporary errors

```

50 ):
51     config = types.GenerateContentConfig(
52         system_instruction=system_instruction,
53         response_mime_type=response_mime_type,
54     )
55
56     # Try the configured model first, then a lightweight fallback.
57     models_to_try = [
58         MODEL_NAME,
59         "gemini-3.5-flash-lite",
60     ]
61
62     # Remove duplicates while keeping order.
63     models_to_try = list(dict.fromkeys(models_to_try))
64
65     last_error = None
66
67     for model in models_to_try:
68         for attempt in range(3):
69             try:
70                 response = client.models.generate_content(
71                     model=model,
72                     contents=prompt,
73                     config=config,
74                 )
75
76                 text = getattr(response, "text", None)
77
78                 if not text:
79                     raise RuntimeError(
80                         "Gemini returned an empty response."
81                     )
82
83             except Exception as exc:
84                 last_error = exc
85
86                 error_text = str(exc).lower()
87
88                 temporary_error = any(
89                     code in error_text
90                     for code in [
91                         "429",
92                         "500",
93                         "503",
94                         "504",
95                         "unavailable",
96                         "overloaded",
97                         "high demand",
98                         "temporarily",
99                     ]
100                 )
101
102                 if not temporary_error:
103                     raise
104
105                 # Exponential backoff: 2s, 4s, 8s
106                 if attempt < 2:
107                     time.sleep(2 ** (attempt + 1))
108
109             return response
110
111     raise last_error
112
113
114
115

```

Fig. ai_client.py – normal text generation

```
50
73
74     for model in models_to_try:
75         for attempt in range(3):
76             except Exception as exc:
77                 if attempt < 2:
78                     time.sleep(2 ** (attempt + 1))
79
80             # Try the fallback model after retries.
81
82             raise RuntimeError(
83                 "Gemini is temporarily unavailable. "
84                 "Please try again in a moment. "
85                 f"Last error: {last_error}"
86             )
87
88
89 def generate_text(
90     prompt: str,
91     *,
92     system_instruction: str | None = None,
93     temperature: float = 0.4,
94     max_output_tokens: int = 2048,
95 ) -> str:
96     """Generate normal text using Gemini."""
97
98     response = _generate_with_retry(
99         prompt,
100         system_instruction=system_instruction,
101         temperature=temperature,
102         max_output_tokens=max_output_tokens,
103     )
```

Fig. ai_client.py – JSON generation and parsing

```
132 -> str:
141
142     text = getattr(response, "text", None)
143
144     if not text:
145         raise RuntimeError("Gemini returned an empty response.")
146
147     return text.strip()
148
149
150 def generate_json(
151     prompt: str,
152     *,
153     system_instruction: str | None = None,
154     max_output_tokens: int = 4096,
155 ) -> Any:
156     """Generate and parse JSON using Gemini."""
157
158     response = _generate_with_retry(
159         prompt,
160         system_instruction=system_instruction,
161         temperature=0.2,
162         max_output_tokens=max_output_tokens,
163         response_mime_type="application/json",
164     )
165
166     text = getattr(response, "text", None)
167
168     if not text:
169         raise RuntimeError(
170             "Gemini returned an empty JSON response."
171         )
```

Fig. ai_client.py – JSON cleanup and decoding

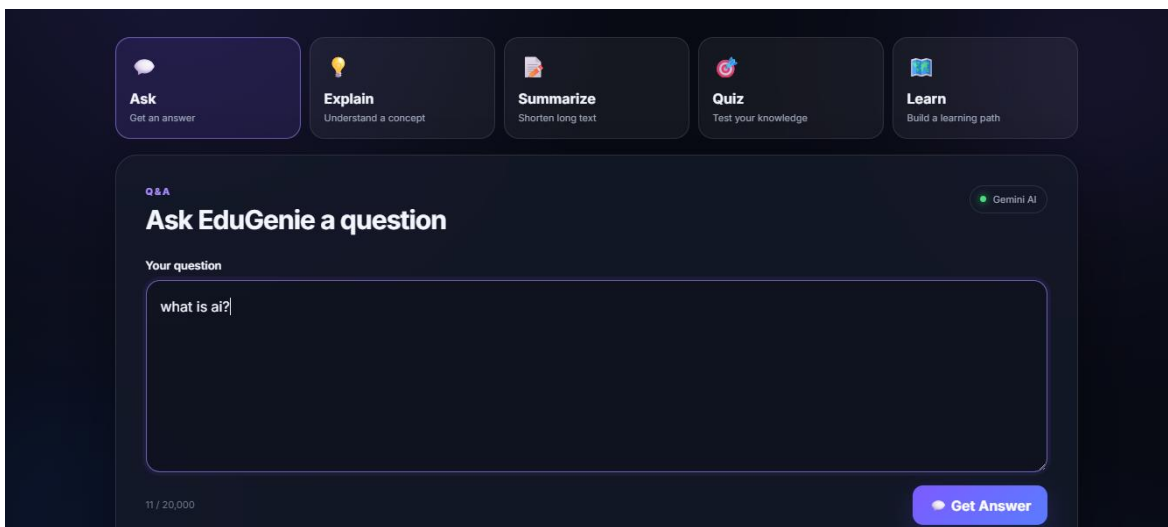
9. Backend API with FastAPI

The current FastAPI backend exposes the following endpoints:

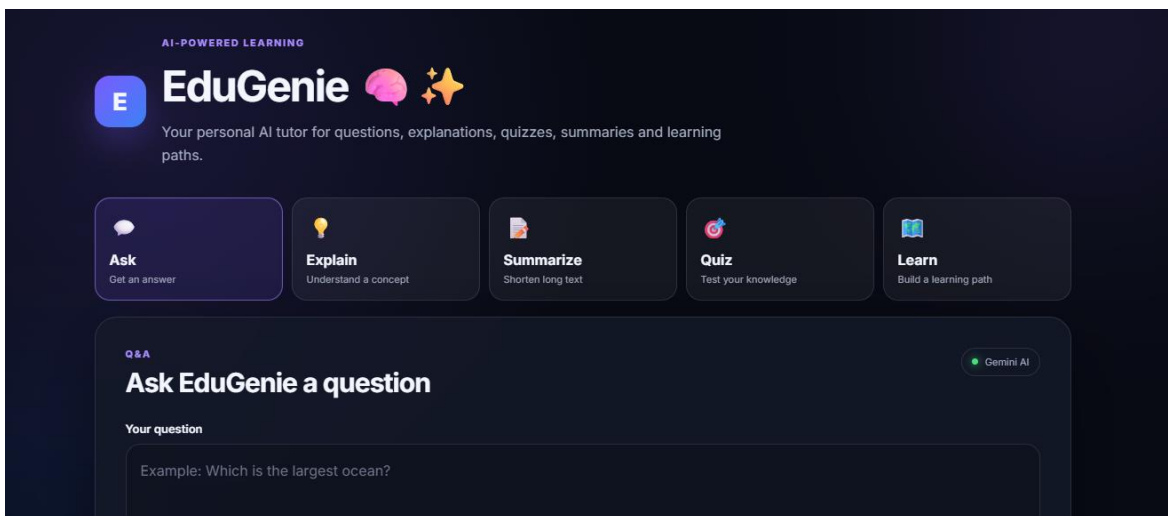
Endpoint	Function
/qa	Question and Answer
/explain	Concept Explanation
/summarize	Text Summarization
/quiz	Quiz Generation
/learn/recommendations	Learning Recommendations

10. Frontend Development

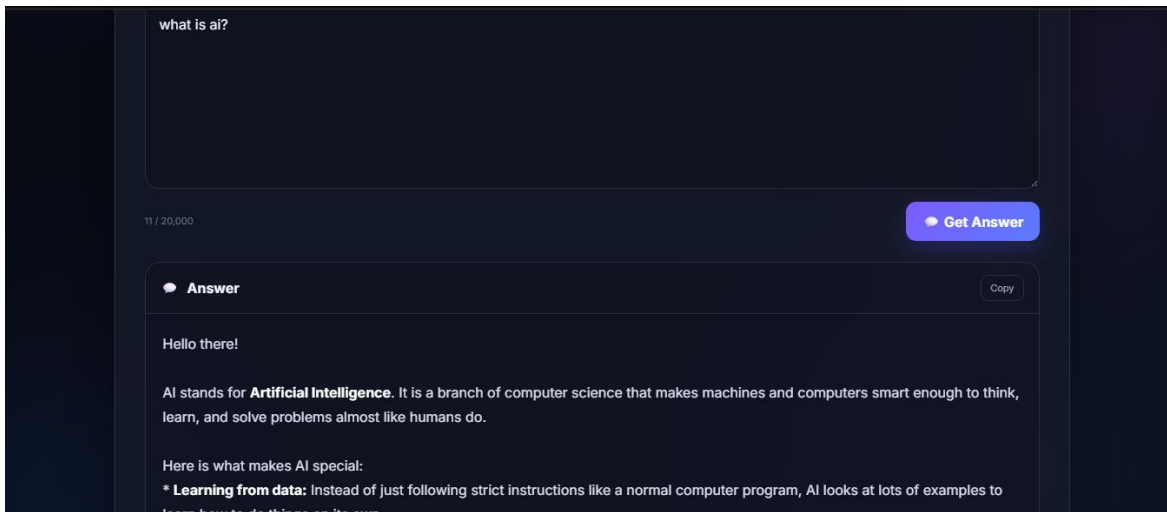
The EduGenie interface provides five primary modes: Ask, Explain, Summarize, Quiz and Learn. The supplied screenshots show the actual working interface and generated responses.



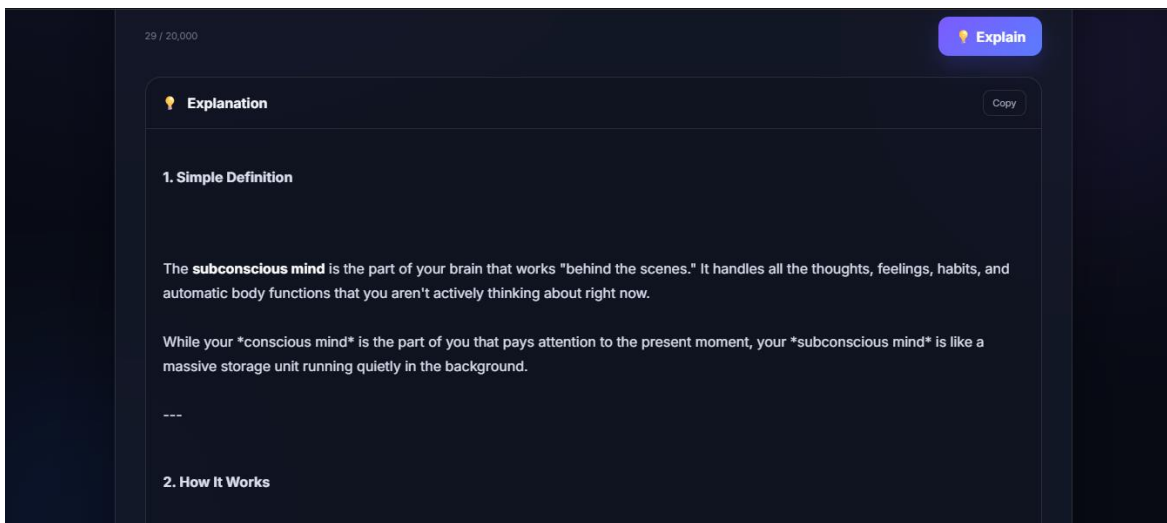
Output 1 – EduGenie Ask interface



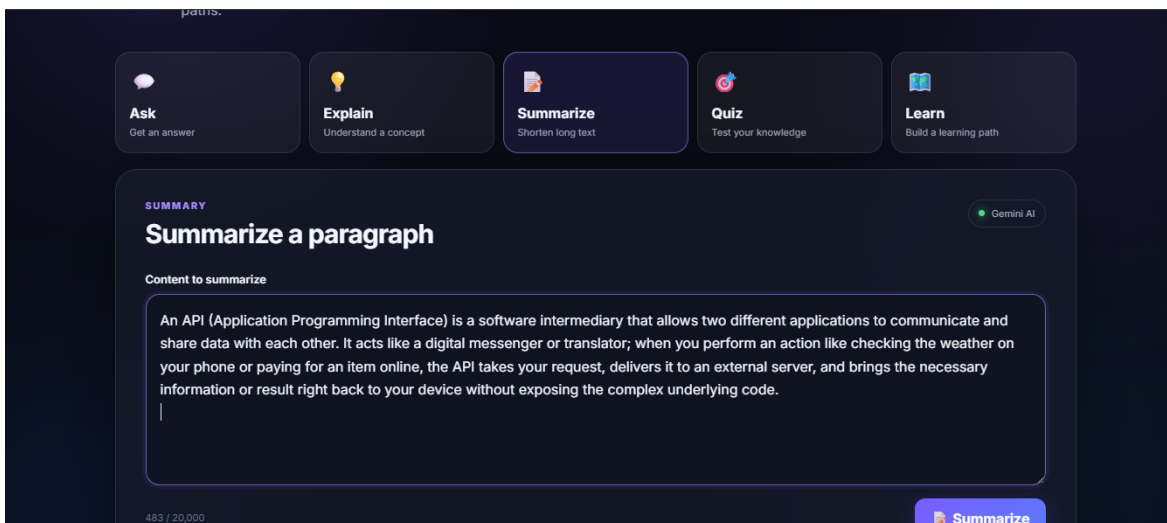
Output 2 – EduGenie home / Ask interface



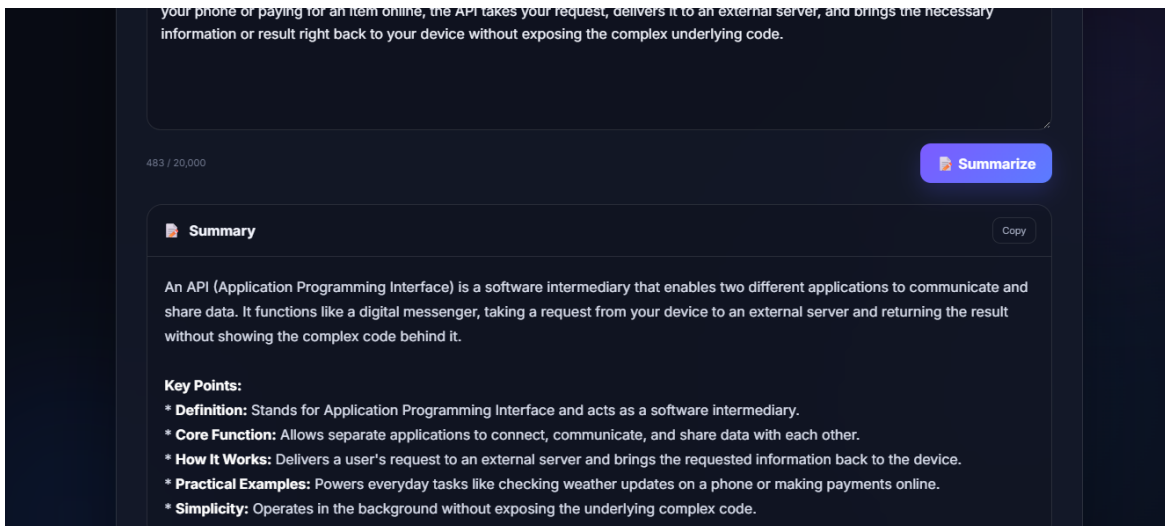
Output 3 – AI answer for the question 'What is AI?'



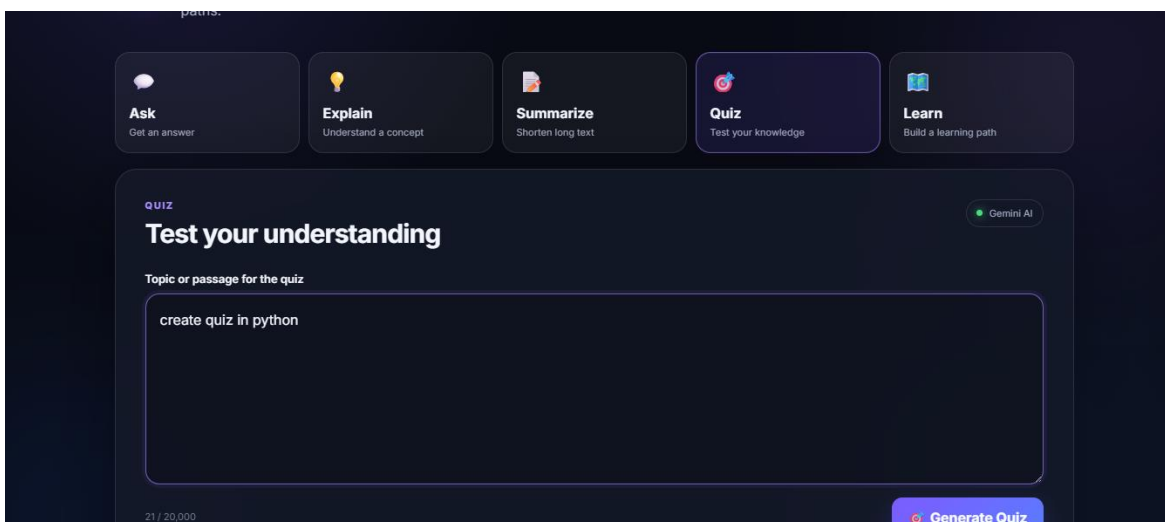
Output 4 – Explain feature output



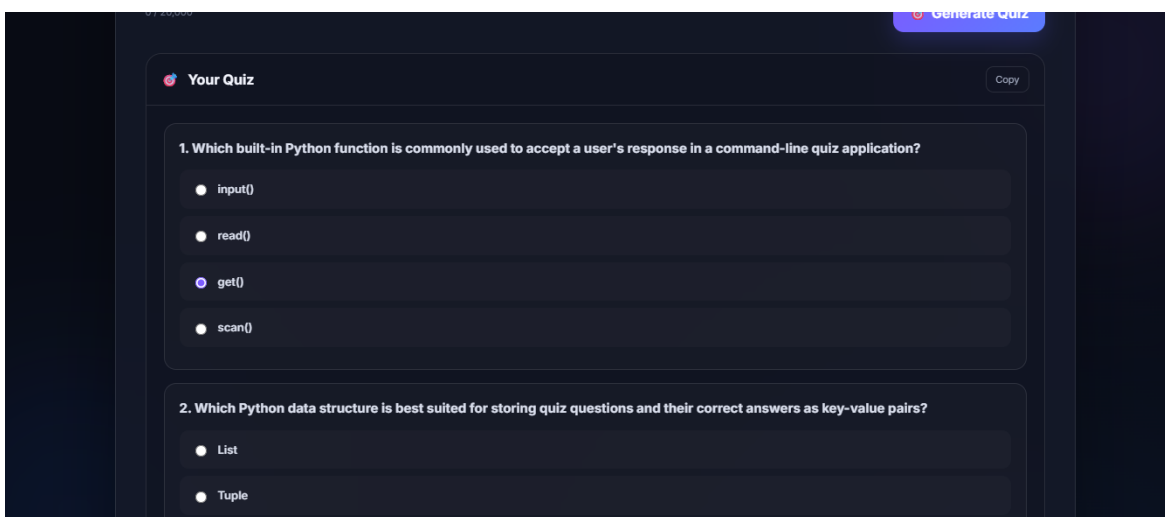
Output 5 – Summarize feature input



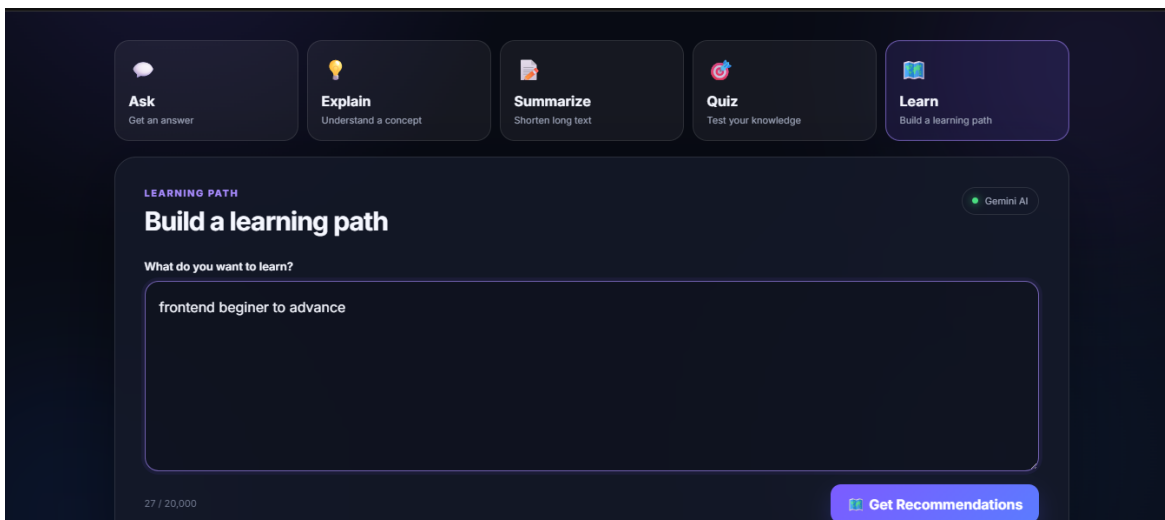
Output 6 – Summary generated by EduGenie



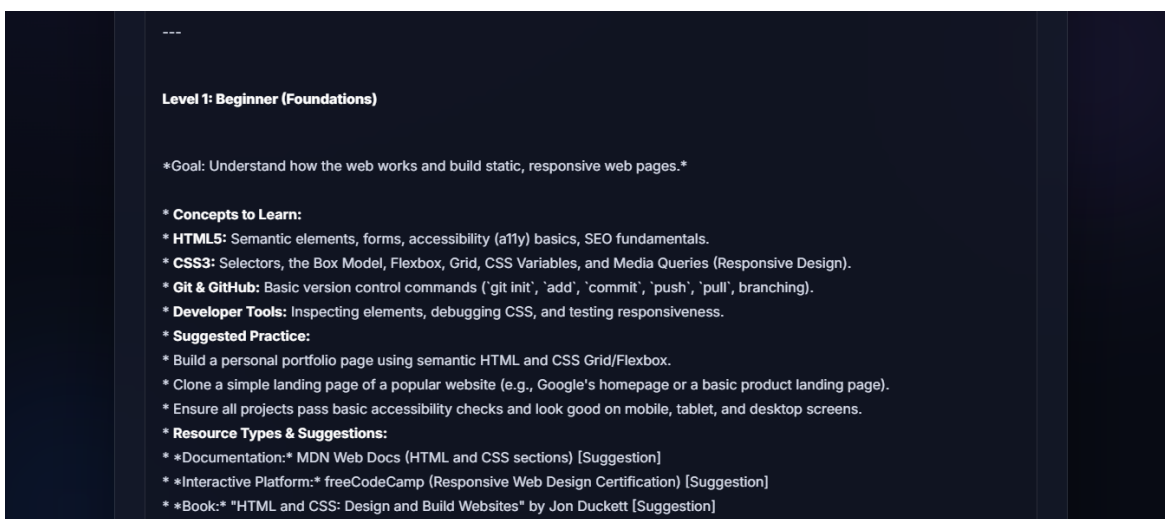
Output 7 – Quiz generation interface



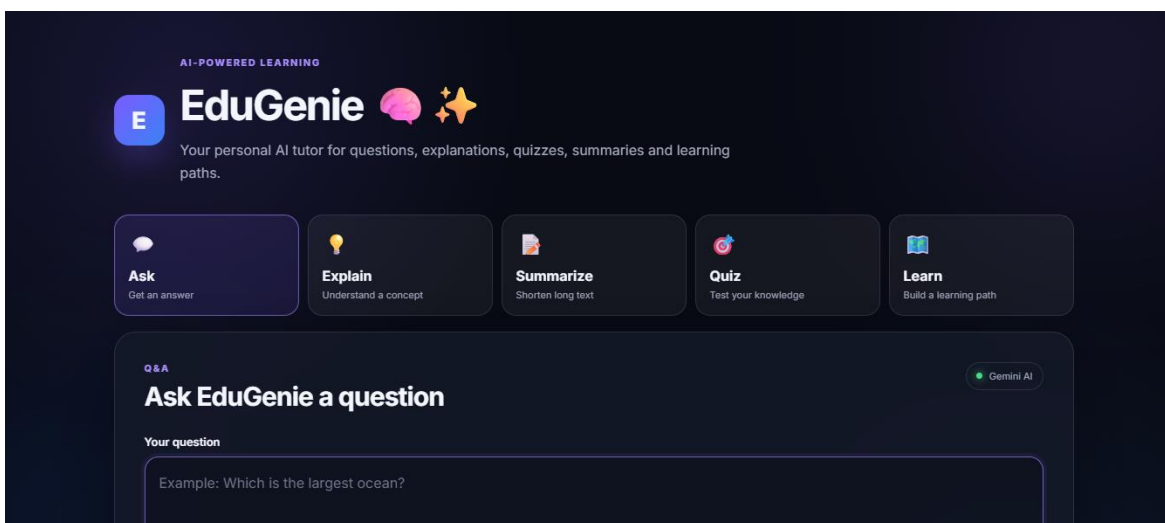
Output 8 – Generated multiple-choice quiz



Output 9 – Learning Path interface



Output 10 – Learning recommendations



Output 11 – EduGenie interface / functional output

11. Deployment and Local Execution

The reference workflow runs the FastAPI application locally with Uvicorn. The supplied project code uses port 8000 for the local server.

```
uvicorn main:app --reload  
  
http://127.0.0.1:8000
```

Fig. Local execution command

12. Functional Testing

Feature	Test Input	Observed Result
Ask	What is AI?	AI answer displayed.
Explain	A selected concept	Structured explanation displayed.
Summarize	Educational paragraph	Concise summary displayed.
Quiz	Create quiz in Python	MCQ quiz displayed.
Learn	Frontend beginner to advance	Learning path displayed.

13. Conclusion

EduGenie: Google Gemini Powered Learning Assistant demonstrates the integration of generative AI with a modular FastAPI web application. The completed system supports question answering, concept explanation, text summarization, quiz generation and personalized learning recommendations. The modular architecture separates the major learning functions, while the shared AI client handles communication with Gemini.

The project provides a practical learning companion through a single web interface and can be extended with additional educational and accessibility features.

14. Future Scope

- Voice-based interaction.
- Multilingual learning support.
- Mobile application and offline capabilities.
- Progress tracking and learning analytics.
- Gamification with badges, streaks and challenges.
- Adaptive learning paths.
- Teacher/parent dashboards and LMS integration.
- Image and PDF based learning inputs.

15. References

- Google AI for Developers – Gemini API documentation.
- FastAPI Documentation.
- Python Documentation.
- Jinja2 Documentation.
- Uvicorn / PyPI documentation.
- Supplied EduGenie project reference document.

16. Submission Details

Submitted By	Anadhavalli A
College	Sri Ram Nallamani Yadava College of Arts and Science
Department	B.Sc. AI & DS
Date	25/09/2026

Mentor: _____